Natural Computing SoSe25

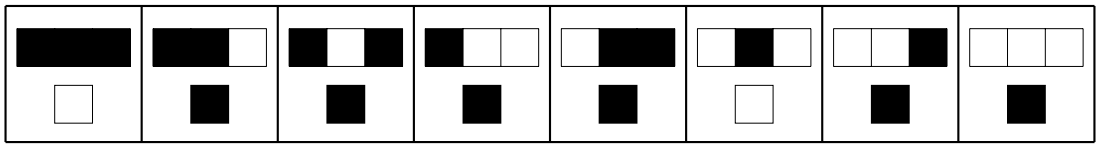
Exercise Sheet 1 (Solution) — May 15, 2025

Thomas Gabor, Maximilian Zorn

1 Cellular Automata

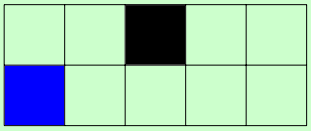
In the lecture we have seen and defined 1D cellular automata (CA) with only two states (`0` or `1`), which evolve over generations according to a predefined rule.

(i) As you might recall from the lecture, a cellular automaton's function f is often given as the template below, which shows the inputs to f on the top row and the respective output on the bottom row. Give a concise mathematical definition for f according to the specific template below.



$$f(l,m,r) = \begin{cases} 0 & \text{if } (l,m,r) \in \{(1,1,1),(0,1,0)\}, \\ 1 & \text{otherwise} \end{cases}$$

(ii) For the following initial configuration, draw one configuration that cannot result from that initial configuration through any cellular automaton ~~following the template of task (i)~~. State your reasoning.



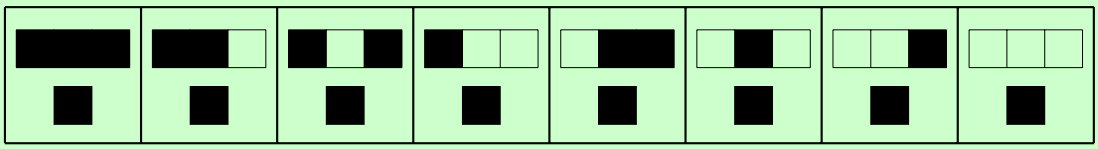
There are two cells which result from (0,0,0) (i.e., the leftmost and the rightmost cell) and, in this case, they have different values, which cannot be generated by a cellular automaton since cellular automata are deterministic.

Wolfram code is a scheme proposed by Stephen Wolfram and the de facto standard for describing elementary cellular automata. A *code* assigns to each template an integer from 0 to 255 as *rule* $n \in [0; 255] \subset \mathbb{N}$ where each digit in n's binary representation encodes one output of f so that the digit for the base value 128 encodes the output of $f(1, 1, 1)$, 64 for $f(1, 1, 0)$, and so on, sorted by decreasing numerical value. This number is taken to be the *rulestring* of the automaton.[1]

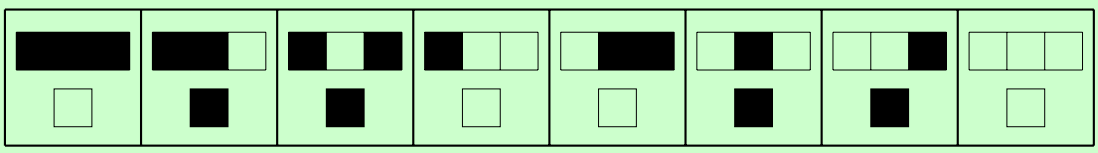
(iii) Given the rulestring `255`, fill in the rule template below. With a three-cell neighborhood, each with two possible states, in how many ways can we configure the rule-template and in how many ways can we set the CA initial configurations for a board of width w?

Possible rule configurations: $2^8 = 256$

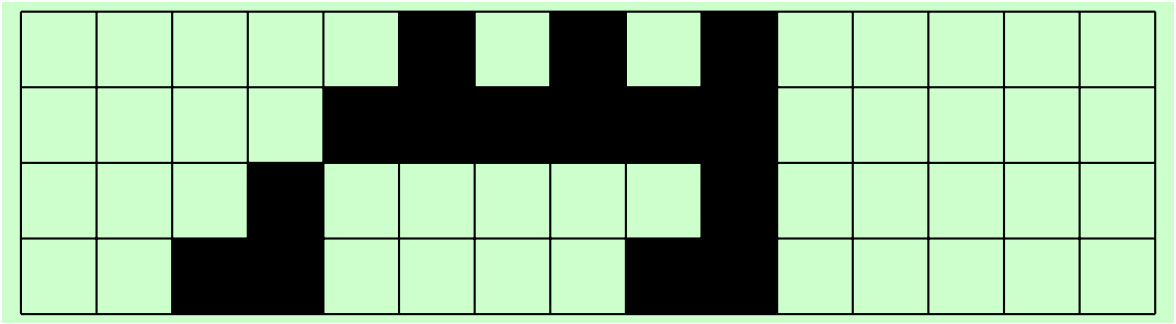
Possible initial board states: 2^w, e.g., $2^{15} = 32768$ in Task (iv)



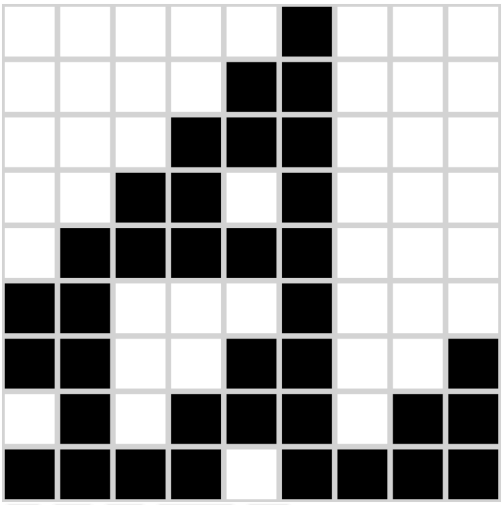
(iv) Fill in the rulestring `102`, then evolve this initial configuration by hand for three generations.



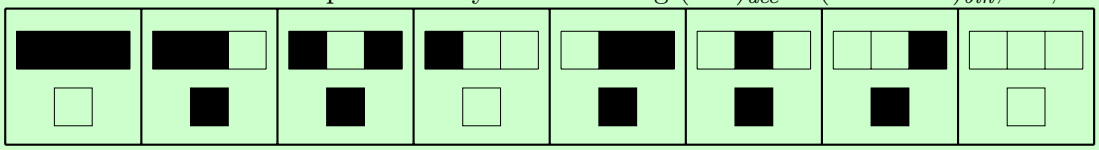
[1] `https://en.wikipedia.org/wiki/Wolfram_code`



(v) Observe the evolution of a new cellular automaton below, showing one generation per row. Give a full definition of this automaton's function f. You may use the empty rule template printed below. Is the given evolution sufficient to fully define f? Why or why not?



The evolution above is represented by the rulestring $(\texttt{110})_{dec} = (\texttt{01101110})_{bin}$, i.e.;



The evolution is sufficient for the definition of f as we can observe the effect of each rule-case at least once.

2 Wolfram's CA Game

Let's put the basics of Task 1 into practice and code a 1d CA evolution game. For a code template consider the included `ca_game.py` file.

(i) In the `*** marked sections ***`, code the rule application and neighborhood check to complete your 1D cellular automaton game. You may consult the respective chapter in "The Nature of Code"[2] if necessary, but you will have to translate Java to Python. If your code is correct, a `pygame` window will open that repeatedly completes the evolution.

For a complete solution that iterates randomly over the 256 valid rule strings refer to `ca_game_solution.py`

(ii) In your rule application, is it possible to directly update the `cells` list in place? What happens if you do so?

No, we cannot overwrite the values in the current *cells* array while we are processing the array, because we need those values to calculate further new values. Otherwise, the next cells in order could not observe their left neighbor at their current generation state.

(iii) (Bonus) *In a cellular automaton, a* ***Garden of Eden*** *is a configuration that has no predecessor. It can be the initial configuration of the automaton but cannot arise in any other way. [..] [F]or any Garden of Eden there is a finite pattern (a subset of cells and their states, called an* ***orphan****) with the same property of having no predecessor, no matter how the remaining cells are filled in. A configuration of the whole automaton is a Garden of Eden if and only if it contains an orphan.*[3]

For 1D cellular automata the existence of Gardens of Eden may be checked by brute force. Change your code to iterate every possible initial state for all 256 rule numbers, keeping track of all states seen per each rule number. (You may reduce the number of cells by increasing `cell_size=50`.) How many evolution steps from every initial state do you need to check for a Garden of Eden? Output for each rule number, how many Gardens of Eden you have found. Why are there trivially no "general Gardens of Eden" that cannot be reached by any rule?

See full code in `garden_of_eden_solution.py`. Here we are iterating through all possible 256 rule patterns and starting states, therefore only needing to observe one evolution for each starting pattern. (This may vary depending on your implementation.) Trivially, there exists one CA that just implements the middle identity function $f(\_, m, \_) = m$, which can reach from the corresponding initial state.

[2] `https://natureofcode.com/book/chapter-7-cellular-automata/`
[3] `https://en.wikipedia.org/wiki/Garden_of_Eden_(cellular_automaton)`